

**Министерство образования, науки и молодежной политики Республики
Коми**

ГПОУ "Сыктывкарский политехнический техникум"

Курсовая работа

Тема: База данных для фотостудии

Выполнил

студент 4 курса

414 группы

Блинников Артём Николаевич

Проверил

Пунгин И.В.

Дата проверки: _____

Задание на курсовую работу по МДК 11.01

"Технология разработки и защиты баз данных"

Специальность:

09.02.07 "Информационные системы и программирование"

Тема курсовой работы:

Разработка базы данных для фотостудии

Срок представления работы к защите: _____

Содержание

- 1. Глава 1**
 - Введение**
 - Цели и задачи**
 - Перечень входных данных**
 - Перечень выходных данных**
 - Ограничения предметной области**
 - Взаимодействие с другими программами**
- 2. Глава 2**
 - Инфологическая (концептуальная) модель базы данных**
 - Определение атрибутов объектов**
 - Определение отношений и кардинальности между объектами**
 - Построение концептуальной модели**
- 3. Глава 3**
 - Логическая структура БД**
- 4. Глава 4**
 - Физическая структура базы данных**
- 5. Глава 5**
 - Реализация проекта в среде конкретного СУБД**
- 6. Заключение**
- 7. Список использованных источников**
- 8. Даты и подписи**

Глава 1

Введение

Современная фотостудия - это сложный бизнес-объект, объединяющий творческие и коммерческие процессы. Основная деятельность включает предоставление услуг фотосъемки (портретной, семейной, свадебной, коммерческой), продажу дополнительных услуг (обработка фотографий, ретушь) и сопутствующих товаров (фотокниги, альбомы), а также управление взаимоотношениями с клиентами.

Ключевые бизнес-процессы, требующие автоматизации:

1. Управление клиентами: Регистрация новых клиентов, ведение истории заказов, отслеживание предпочтений и контактной информации.
2. Оформление и учет заказов: Создание заказов, расчет стоимости, выбор услуг, фиксация способов оплаты и связи.
3. Управление услугами и ценами: Ведение актуального каталога услуг с гибким ценообразованием.
4. Учет сотрудников: Хранение информации о фотографах, администраторах, ретушерах и их должностях.
5. Финансовая отчетность: Формирование чеков, ежедневных отчетов о выручке, анализ популярности услуг и клиентской активности.
6. Контроль доступа: Разграничение прав сотрудников на основе их ролей в системе (владелец, администратор, работник, бухгалтер).

Проблемы, решаемые разрабатываемой системой:

1. Дублирование и потеря данных при ведении учета в бумажных журналах и разрозненных электронных таблицах.
2. Ошибки в расчетах стоимости заказов при ручном подсчете.
3. Отсутствие единой клиентской базы, что затрудняет анализ покупательского поведения и проведение маркетинговых кампаний.
4. Сложность формирования отчетов для анализа эффективности работы студии.
5. Отсутствие системы контроля доступа к коммерческой и клиентской информации.

Цель проекта

Разработка интегрированной информационной системы на основе реляционной базы данных и клиентского приложения для комплексной автоматизации операционной деятельности фотостудии, повышения точности учета, скорости обслуживания клиентов и качества управленческих решений.

Задачи, решаемые системой:

1. Автоматизация процессов приема и обработки заказов.
2. Создание единой электронной базы клиентов с историей взаимодействий.
3. Реализация механизма формирования финансовых и аналитических отчетов.
4. Внедрение ролевой модели разграничения прав доступа (RBAC).
5. Обеспечение целостности, безопасности и сохранности данных.
6. Предоставление удобного графического интерфейса для пользователей с разным уровнем подготовки.

Перечень входных данных

Система обрабатывает данные, поступающие от пользователей через графический интерфейс и поступающие из внешних источников. Все входные данные можно классифицировать по нескольким категориям.

1. Справочная информация (нормативные данные)

1.1. Данные о должностях (post)

- post_id - Уникальный ID (Первичный ключ)
- name - Наименование должности (текстовое поле, обязательное, уникальное).

Содержание таблицы: "owner", "admin", "worker", "accountant".

Источник: Ввод администратором или владельцем через интерфейс управления справочниками.

1.2. Данные о сотрудниках (employee)

- employee_id - Уникальный ID (Первичный ключ)
- surname - Фамилия сотрудника (текстовое поле, обязательное).
- name - Имя сотрудника (текстовое поле, обязательное).

- patronymic - Отчество сотрудника (текстовое поле, необязательное).
- post_id - Идентификатор должности (целочисленное, связь с таблицей post).
- phone - Контактный телефон (строковое поле).

Источник: Ввод администратором при приеме на работу или обновлении данных.

1.3. Данные о способах оплаты (payment_method)

- payment_method_id - Уникальный ID (Первичный ключ)
- name - Название способа оплаты (текстовое поле, обязательное, уникальное).

Содержание таблицы: "Наличные", "Банковская карта", "Перевод".

Источник: Предусмотренные значения, может редактировать администратор.

1.4. Данные о способах связи (contact_method)

- contact_method_id - Уникальный ID (Первичный ключ)
- name - Название способа связи (текстовое поле, обязательное, уникальное).

Содержание таблицы: "Телефон", "Telegram", "WhatsApp".

Источник: Предусмотренные значения, может редактировать администратор.

1.5. Данные об услугах (service)

- service_id - Уникальный ID (Первичный ключ)
- name - Наименование услуги (текстовое поле, обязательное).
- service_price - Стоимость услуги (числовое поле, точность 10 знаков, 2 после запятой, обязательное).

Источник: Ввод и регулярное обновление администратором или владельцем.

2. Операционные данные (транзакционные) 2.1.

Данные о клиентах (client)

- client_id - Уникальный ID (Первичный ключ)
- surname - Фамилия клиента (текстовое поле, необязательное).
- name - Имя клиента (текстовое поле, обязательное).
- patronymic - Отчество клиента (текстовое поле, необязательное).
- phone - Контактный телефон (строковое поле, основной идентификатор, необязательное).
- email - Адрес электронной почты (строковое поле, необязательное).

Источник: Ввод работником (администратором, фотографом) при первом обращении клиента или самим клиентом через веб-форму (в перспективе).

2.2. Данные о заказах (orders)

- order_id - Уникальный ID (Первичный ключ)
- client_id - Идентификатор клиента (целочисленное, обязательное, связь с таблицей client).
- payment_method_id - Идентификатор способа оплаты (целочисленное, обязательное, связь с таблицей payment_method).
- date - Дата и время создания заказа (метка времени, заполняется автоматически).
- total_sum - Итоговая сумма заказа (вычисляемое поле на основе данных из order_info).
- contact_method_id - Идентификатор способа связи (целочисленное, обязательное).

Источник: Формируется автоматически при оформлении заказа работником через интерфейс "Корзина".

2.3. Детализация заказов (order_info)

- order_info_id - Уникальный ID (Первичный ключ)
- order_id - Идентификатор заказа (целочисленное, связь с таблицей orders).

- service_id - Идентификатор услуги (целочисленное, связь с таблицей service).
- quantity - Количество единиц данной услуги в заказе (целочисленное, >0).

Источник: Добавление услуг в "Корзину" работником при оформлении заказа.

3. Административные данные

3.1. Данные пользователей системы (users)

- user_id - Уникальный ID (Первичный ключ)
- username - Логин для входа в систему (текстовое поле, обязательное, уникальное).
- password_hash - Пароль (в открытом виде на входе, далее хранится хэш).
- role - Роль в системе (выбор из списка: 'owner', 'admin', 'worker', 'accountant').
- name - Отображаемое имя пользователя (текстовое поле).

Источник: Создание учетной записи владельцем или администратором через панель управления.

4. Параметры и данные сессии

4.1. Данные текущей сессии (временные)

- Корзина заказа: Список выбранных услуг (service_id) и их количество (quantity) до оформления.
- Параметры фильтрации отчетов: Выбранные даты, типы услуг, клиенты для формирования отчетов.
- Учетные данные для входа: Логин и пароль, введенные в форме авторизации (проверяются, но не хранятся).

Источник: Действия пользователя в графическом интерфейсе.

4.2. Данные для импорта/экспорта

- Файлы в формате .csv или .xlsx со списком клиентов или услуг для массовой загрузки (в перспективе).
- Источник: Внешние системы (например, CRM, сайт-визитка) или архивные данные.

Сводная таблица входных данных:

Категория	Сущность	Ключевые поля	Источник	Обязательность
Справочная	post	name	Администратор Владелец	Да
	employee	surname, name, post_id	Администратор	Да
	payment_method	name	Система Администратор	Да
	contact_method	name	Система Администратор	Да
	service	Name, service_price	Администратор владелец	Да
Операционная	client	Name, phone	Работник Клиент	Да
	orders	client_id, payment_method_id	Система (на основе корзины)	Да
	order_info	order_id, service_id, quantity	Работник (добавление в корзину)	Да
Административная	users	username, password, role	Владелец Администратор	Да
Сессионная	Корзина	service_id, quantity	Работник	Нет (временные)

1.3.Перечень выходных данных

Система генерирует выходные данные в различных форматах для поддержки операционной деятельности, финансового учета и управленческого анализа. Выходные данные формируются на основе обработки входной информации и выполнения бизнес-логики.

1. Операционные документы (для клиентов и внутреннего учета)

1.1. Электронный чек заказа

Формат: Текстовый файл (.txt), структурированный для печати на фискальном аппарате или обычном принтере.

Содержание:

- Шапка: Название фотостудии, контактные данные.
- Номер заказа и дата/время оформления.
- Данные клиента (ФИО, контакт).
- Детализация заказа: список услуг с количеством, ценой за единицу и суммой по позиции.
- Итоговая сумма к оплате.
- Выбранные способ оплаты и связи.
- Подпись и печать (шаблон).

Назначение: Подтверждение совершения сделки для клиента, основание для внесения оплаты, документ для внутреннего архива.

Пример имени файла: чек_заказа_105_20241215.txt

1.2. Счет на оплату (в перспективе) Формат:

PDF-документ.

Содержание: Реквизиты студии, реквизиты клиента (для юр. лиц), детализация услуг, итоговая сумма, срок оплаты.

Назначение: Формальный документ для безналичных расчетов с юридическими лицами.

2. Отчетные формы (для внутреннего анализа)

2.1. Ежедневный отчет о продажах

Формат: Текстовое представление в интерфейсе системы, с возможностью экспорта в CSV/TXT.

Содержание:

- Отчетная дата.

- Сводные показатели: количество заказов, общая выручка, количество уникальных клиентов, средний чек.
- Детализация по услугам: наименование, количество продаж, выручка по каждой услуге.
- Список всех заказов дня с основной информацией (номер, клиент, сумма, способ оплаты).
- Назначение: Контроль ежедневной выручки, оперативный анализ активности. Основной инструмент для администратора в конце рабочего дня.

Генерация: По запросу пользователя с возможностью выбора даты.

2.2. Финансовый отчет за период

Формат: Интерактивная таблица в интерфейсе, с возможностью экспорта в Excel (.xlsx) и CSV (.csv).

Содержание:

- Период (начальная и конечная дата).
- Детализированный список всех заказов за период с полной информацией (ID, дата, клиент, состав услуг, сумма, способ оплаты).
- Итоговые суммы по дням/неделям/месяцам (в зависимости от группировки).

Назначение: Подготовка данных для бухгалтерского учета, детальный анализ продаж за любой период. Основной инструмент для бухгалтера и владельца.

Генерация: По запросу с настройкой фильтров (дата, клиент, услуга, способ оплаты).

2.3. Аналитический отчет (статистика)

Формат: Текстовый вывод в специальной вкладке интерфейса.

Содержание:

- Топ-N услуг: Ранжированный список услуг по количеству продаж и общей выручке.

- Топ-N клиентов: Ранжированный список клиентов по количеству заказов и общей сумме покупок.
- Динамика продаж: Сравнение показателей за разные периоды (например, выручка в этом месяце vs. прошлый месяц).

Назначение: Стратегический анализ бизнеса, выявление популярных услуг и наиболее ценных клиентов. Инструмент для владельца и администратора для планирования.

Генерация: По запросу пользователя.

3. Справочники и выборки (для работы в системе)

3.1. Список клиентов

Формат: Интерактивная таблица в интерфейсе с возможностью сортировки и фильтрации.

Содержание: Все поля таблицы client (ФИО, телефон, email). При выборе клиента для нового заказа отображается его краткая история (последние заказы).

Назначение: Быстрый поиск и выбор клиента при оформлении заказа, просмотр контактов.

3.2. Список доступных услуг

Формат: Выпадающий список или отдельная таблица в интерфейсе оформления заказа.

Содержание: Наименование услуги и ее текущая цена.

Назначение: Выбор услуг для добавления в корзину заказа.

3.3. График/календарь (в перспективе)

Формат: Графическое представление (дни/недели) в интерфейсе.

Содержание: Забронированные временные слоты для фотосессий с привязкой к заказам, клиентам и фотографам.

Назначение: Визуальное планирование загрузки студии и сотрудников.

4. Системные данные

4.1. Резервная копия базы данных

Формат: SQL-дамп (.sql или .dump), сжатый архив.

Содержание: Полный снимок всех таблиц и данных базы photo_studio на момент создания.

Назначение: Восстановление системы в случае сбоя, миграция на новый сервер. Создается по запросу владельца.

Пример имени файла: backup_photo_studio_20241215_full.sql.gz

4.2. Журнал аудита (логи действий)

Формат: Текстовый файл или записи в специальной таблице БД.

Содержание: Хронологическая запись критических действий пользователей (вход в систему, создание/изменение/удаление заказов, изменение справочников) с указанием пользователя, времени и сути изменения.

Назначение: Обеспечение безопасности и отслеживаемости. Для анализа в спорных ситуациях.

Сводная таблица выходных данных:

Категория	Название документа \отчета	Формат	Основной потребитель	Назначение
Операционные	Чек заказа	TXT, CSV	Клиент, Работник	Подтверждение сделки
	Счет на оплату	PDF (перспектива)	Клиент (юр. лицо)	Для безналичного расчета
Отчетные	Ежедневный отчет о продажах	TXT/CSV (в интерфейсе)	Администратор Владелец	Оперативный контроль
	Финансовый отчет за период	XLSX, CSV, Интерфейс	Бухгалтер Владелец	Бухгалтерский учет, анализ
	Аналитический отчет (статистика)	Текст в интерфейсе	Владелец Администратор	Стратегическое планирование
Справочные	Список клиентов	Интерактивная таблица	Все роли	Поиск и выбор клиента
	Список услуг	Выпадающий список	Работник Администратор	Оформление заказа
Системные	Резервная копия БД	SQL, GZIP	Владелец Админ. БД	Восстановление данных

	Журнал аудита	ТХТ/Таблица БД	Владелец	Контроль безопасности
--	---------------	----------------	----------	-----------------------

1.4.Ограничения предметной области

Разрабатываемая система имеет ряд ограничений, которые определяют границы ее функциональности и область применения. Эти ограничения сформулированы на основе анализа реальных потребностей фотостудии и технических возможностей реализации.

1. Функциональные ограничения

1.1. Ограничения по бизнес-процессам:

- Система не управляет расписанием (бронированием времени). Не предусмотрены функции календарного планирования фотосессий, бронирования студий или оборудования. Учет времени фотографов и помещений ведется вне системы.
- Отсутствует модуль управления задачами (Task Management). Система не назначает автоматически задачи сотрудникам (например, "обработать заказ №Х к дате Y") и не отслеживает их выполнение.
- Нет управления складскими запасами материальных товаров. Учитываются только услуги. Продажа физических товаров (фотобумага, рамки, альбомы) реализована через услугу с фиксированной ценой без контроля остатков на складе.
- Отсутствует система скидок и промо-акций. Расчет стоимости ведется строго по прайс-листу. Нет механизмов для накопительных скидок, подарочных сертификатов или сезонных акций.

1.2. Ограничения по взаимодействию с клиентом:

Нет клиентского личного кабинета (веб- или мобильного). Взаимодействие с клиентом инициируется исключительно сотрудником студии через desktopприложение.

Отсутствует онлайн-запись. Клиент не может самостоятельно выбрать свободное время и забронировать сессию через сайт или приложение.

Нет интеграции с системами онлайн-оплаты. Фиксируется только факт и способ оплаты (наличные, карта). Проведение платежа через эквайринг, онлайн-банки или платежные агрегаторы происходит вне системы.

Не реализованы автоматические уведомления (SMS/Email). Напоминания клиентам о предстоящей съемке или готовности заказа отправляются вручную.

2. Технические ограничения

2.1. Ограничения по данным:

- Не поддерживается хранение файлов изображений. База данных хранит только структурированные метаданные (о клиентах, заказах, услугах). Сами файлы фотографий высокого разрешения хранятся в файловой системе или облачных хранилищах. В БД могут сохраняться только ссылки на них.
- Ограниченная работа с историческими ценами. Система фиксирует итоговую сумму заказа, но не сохраняет "историю цен" на услуги в момент каждого заказа. Это может усложнить ретроспективный анализ маржинальности, если цены на услуги менялись.

2.2. Ограничения архитектуры и масштабируемости:

- Однопользовательская архитектура на уровне транзакций. Хотя к системе могут подключаться несколько пользователей, сложные конкурентные операции (например, попытка двух администраторов одновременно изменить цену одной и той же услуги) могут требовать дополнительной реализации механизмов блокировок.
- Предназначена для одной точки работы. Система не рассчитана на распределенную сеть филиалов с единой централизованной базой данных и синхронизацией между ними.
- Desktop-ориентированность. Основное клиентское приложение — это программа для Windows/Linux/macOS. Нет веб-интерфейса или мобильной версии для базовых операций.

3. Ограничения по интеграции

- Нет готовых интеграций со сторонними сервисами. Система работает изолированно. Интеграция с бухгалтерскими программами (1С), службами

доставки, сервисами email-рассылок или CRM требует дополнительной разработки модулей или использования ручного экспорта/импорта данных (CSV, Excel).

- Нет API для внешних систем в базовой версии. Внешние приложения (например, сайт студии) не могут напрямую запрашивать данные (свободное время, цены) или создавать заказы без разработки дополнительного программного интерфейса.

4. Административные ограничения

- Ручное управление резервным копированием. Процесс создания и проверки резервных копий инициируется администратором вручную через соответствующий интерфейс, а не выполняется полностью автоматически по расписанию.
- Базовый уровень аудита. Система логирует основные действия (создание/удаление заказов, вход пользователей), но может не детализировать все изменения (например, кто и когда изменил конкретное поле в карточке клиента).

Резюме ограничений:

Аспект	Ограничение	Следствие/Компенсация
Функции	Нет календаря бронирования	График ведется в сторонних календарях (Google, Яндекс)
Функции	Нет учета товарных остатков	Товары учитываются как услуги, остатки - вручную
Данные	Не хранятся файлы фото	Фото хранятся в папках/облаке, в БД - только ссылки
Архитектура	Для одной студии, desktop	Оптимально для малого бизнеса, простота развертывания
Интеграция	Нет готовых интеграций	Обмен данными через экспорт в CSV/XLSX
Масштаб	10-15 одновременных пользователей	Достаточно для штата типичной студии

Данные ограничения сознательно приняты на этапе проектирования для создания минимально жизнеспособного продукта (MVP), который решает ключевые проблемы учета и отчетности, оставаясь в рамках учебного проекта и разумных сроков разработки. Большинство из этих ограничений может быть снято в последующих версиях системы.

1.5. Взаимодействие с другими программами

Система не существует в изоляции и взаимодействует с другим программным обеспечением на различных уровнях: от системного до прикладного. Это взаимодействие обеспечивает ее работоспособность, расширяет функциональность и позволяет интегрироваться в существующую ИТ-инфраструктуру.

1. Системное программное обеспечение (обязательное)

1.1. Система управления базами данных (СУБД):

Программа: PostgreSQL (версия 14 или выше).

Характер взаимодействия: Прямое, критически важное.

Роль: Основное хранилище всех структурированных данных системы. Клиентское приложение через драйвер psycopg2 отправляет SQL-запросы (SELECT, INSERT, UPDATE, DELETE) и получает результаты.

Используемые протоколы/интерфейсы: Сетевое соединение по TCP/IP, стандартный протокол PostgreSQL.

Данные для обмена: Все данные предметной области (таблицы clients, orders, services и т.д.), а также системные настройки соединения (хост, порт, имя БД, логин, пароль).

1.2. Интерпретатор языка программирования:

Программа: Python (версия 3.8 или выше).

Характер взаимодействия: Прямое, критически важное.

Роль: Среда выполнения для клиентского приложения. Все бизнес-логика и код интерфейса написаны на Python.

Используемые библиотеки (зависимости):

- psycopg2 — драйвер для соединения с PostgreSQL.
- PyQt6 — фреймворк для создания графического пользовательского интерфейса (GUI).
- pandas — библиотека для обработки данных и экспорта отчетов в Excel.
- Стандартные библиотеки: hashlib (для шифрования паролей), csv, datetime.

2. Прикладное программное обеспечение (опциональное/вспомогательное)

2.1. Офисные пакеты:

Программы: Microsoft Excel, LibreOffice Calc, Google Sheets.

Характер взаимодействия: Косвенное, через файлы.

Роль: Просмотр, дополнительная обработка и печать отчетов, сформированных системой.

Используемые форматы: .xlsx (Microsoft Excel Open XML), .csv (CommaSeparated Values).

Сценарий: Пользователь (бухгалтер, владелец) формирует в системе "Финансовый отчет за период" и экспортирует его в файл **report_2024_12.xlsx**. Этот файл затем открывается в Excel для создания сводных таблиц, построения графиков или подготовки презентации.

2.2. Текстовые редакторы и программы для печати:

Программы: Блокнот (Notepad), Notepad++, любой другой текстовый редактор, программа для печати текстовых файлов.

Характер взаимодействия: Косвенное, через файлы.

Роль: Просмотр и печать электронных чеков, сгенерированных системой.

Используемые форматы: .txt (простой текст).

Сценарий: После оформления заказа система создает файл чек_заказа_105.txt. Этот файл можно открыть для проверки, а затем отправить на печать на подключенный принтер, в том числе на фискальный регистратор, поддерживающий печать из текстового файла.

2.3. Файловые менеджеры и облачные хранилища:

Программы: Проводник Windows, Finder macOS, Nautilus Linux; Dropbox, Яндекс.Диск, Google Drive.

Характер взаимодействия: Косвенное, через файловую систему.

Роль: Организация хранения и архивации выходных документов (чеков, отчетов, резервных копий).

Сценарий: Резервная копия БД (backup_photo_studio.sql.gz) вручную копируется администратором в папку облачного хранилища для защиты от потери данных в случае поломки локального сервера.

2.4. Почтовые клиенты и мессенджеры:

Программы: Outlook, Thunderbird, Gmail; Telegram, WhatsApp.

Характер взаимодействия: Ручное, через буфер обмена или вложение файла.

Роль: Коммуникация с клиентами. Отправка чеков, обсуждение деталей заказа.

Сценарий: Администратор копирует содержание чека или прикрепляет файл отчета и вручную отправляет клиенту через email или мессенджер, указанный как предпочтительный способ связи (contact_method).

3. Внешние системы и API (потенциальное взаимодействие)

3.1. Веб-сайт фотостудии:

Характер взаимодействия: Потенциальное, через API (см. пункт 5.8).

Роль: Получение данных для отображения на сайте.

Возможные сценарии:

Вывод на странице "Услуги" актуального прайс-листа, загруженного из БД.

Отображение на странице "Фотографы" информации о сотрудниках из таблицы employees.

Прием предварительных заявок с сайта и их сохранение в БД как черновиков заказов.

3.2. Бухгалтерские системы (1С, БухСофт и аналоги):

Характер взаимодействия: Косвенное, через файлы.

Роль: Передача финансовых данных для официального учета.

Сценарий: Ежемесячный отчет по закрытым заказам и оплатам экспортируется из системы в формате .csv с определенной структурой столбцов, которая затем загружается в конфигурацию 1С "Бухгалтерия предприятия".

3.3. СМС-шлюзы и сервисы email-рассылок:

Характер взаимодействия: Потенциальное, через API.

Роль: Автоматизация уведомлений клиентов.

Сценарий (перспектива): При смене статуса заказа на "Готов" система автоматически отправляет HTTP-запрос к API сервиса SMS.ru или SendGrid для отправки клиенту уведомления.

Итог: Система построена по классической клиент-серверной архитектуре с использованием стабильных и распространенных технологий (Python, PostgreSQL, PyQt). Ее основное и критическое взаимодействие — с СУБД PostgreSQL. Вспомогательное взаимодействие с офисными пакетами и файловыми менеджерами осуществляется через обмен файлами стандартных форматов, что обеспечивает простоту и надежность. Потенциал для глубокой интеграции со сторонними сервисами (сайт, CRM, SMS) заложен в архитектуре и может быть реализован в будущем.

Глава 2

2. Инфологическая (концептуальная) модель базы данных

Концептуальная модель представляет предметную область в виде совокупности информационных объектов (сущностей) и связей между ними, независимо от особенностей конкретной СУБД. Это "мост" между бизнеспроцессами фотостудии и технической реализацией базы данных.

2.1. Выделение объектов информации

На основе анализа процессов (раздел 1.1) и перечня данных (разделы 1.2, 1.3) были выделены следующие ключевые сущности (объекты информации):

- Клиент (**Client**) — центральная сущность. Физическое или юридическое лицо, пользующееся услугами студии. Каждый клиент уникален и может совершать множество заказов.
- Сотрудник (**Employee**) — работник фотостудии. Имеет должность и контактные данные. Сущность важна для внутреннего учета и

потенциального расширения функционала (привязка заказов к конкретному фотографу).

- Должность (**Post**) — справочная сущность. Определяет роль сотрудника в организации (фотограф, ретушер, администратор). Позволяет структурировать штатное расписание.
- Услуга (**Service**) — основная продаваемая единица. Описывает конкретную работу или продукт студии (например, "часовая фотосессия в студии", "ретушь 10 фотографий") с фиксированной ценой.
- Способ оплаты (**Payment_Method**) — справочная сущность. Фиксирует возможные варианты расчета с клиентом (наличные, карта, перевод). Необходима для финансового учета и формирования чека.
- Способ связи (**Contact_Method**) — справочная сущность. Определяет предпочтительный канал коммуникации с клиентом (телефон, email, мессенджер). Важен для сервиса и маркетинга.
- Заказ (**Order**) — ключевая операционная сущность. Фактический документ, фиксирующий сделку между студией и клиентом. Связывает клиента, выбранные им услуги и условия сделки (оплата, связь) в конкретный момент времени.
- Состав заказа (**Order_Info**) — вспомогательная сущность, реализующая сложную связь. Описывает, какие конкретно услуги и в каком количестве входят в один заказ. Позволяет одному заказу содержать несколько позиций.
- Пользователь системы (**User**) — служебная сущность. Учетная запись для доступа к программному обеспечению. Не связана напрямую с бизнеспроцессами, но критична для безопасности и работы приложения.

2.2. Определение атрибутов объектов

Для каждой сущности определен полный набор атрибутов (характеристик, свойств).

Сущность: Клиент (Client)

- client_id - Уникальный идентификатор клиента. Тип: Целочисленный, автоинкрементный.
- surname - Фамилия. Тип: Текстовый.

- name - Имя. Тип: Текстовый, обязательный.
- patronymic - Отчество. Тип: Текстовый, необязательный.
- phone - Основной контактный телефон. Тип: Строковый (VARCHAR).
- email - Адрес электронной почты. Тип: Текстовый.

Сущность: Сотрудник (Employee)

- employee_id - Уникальный идентификатор сотрудника. Тип: Целочисленный, автоинкрементный.
- surname - Фамилия. Тип: Текстовый, обязательный.
- name - Имя. Тип: Текстовый, обязательный.
- patronymic - Отчество. Тип: Текстовый.
- post_id - Ссылка на должность. Тип: Целочисленный.
- phone - Контактный телефон. Тип: Строковый (VARCHAR).

Сущность: Должность (Post)

- post_id - Уникальный идентификатор должности. Тип: Целочисленный, автоинкрементный.
- name - Наименование должности. Тип: Текстовый, обязательный, уникальный.

Сущность: Услуга (Service)

- service_id - Уникальный идентификатор услуги. Тип: Целочисленный, автоинкрементный.
- name - Наименование услуги. Тип: Текстовый, обязательный.
- service_price - Цена услуги в рублях. Тип: Числовой (NUMERIC), точность 10 знаков, 2 знака после запятой, обязательный.

Сущность: Способ оплаты (Payment_Method)

- payment_method_id - Уникальный идентификатор способа. Тип: Целочисленный, автоинкрементный.
- name - Наименование способа. Тип: Текстовый, обязательный, уникальный.

Сущность: Способ связи (Contact_Method)

- contact_method_id - Уникальный идентификатор способа. Тип: Целочисленный, автоинкрементный.
- name - Наименование способа. Тип: Текстовый, обязательный, уникальный.

Сущность: Заказ (Order)

- order_id - Уникальный идентификатор заказа. Тип: Целочисленный, автоинкрементный.
- client_id - Ссылка на клиента. Тип: Целочисленный, обязательный.
- payment_method_id - Ссылка на способ оплаты. Тип: Целочисленный, обязательный.
- contact_method_id - Ссылка на способ связи. Тип: Целочисленный, обязательный.
- date - Дата и время создания заказа. Тип: Временная метка (TIMESTAMP), заполняется автоматически.
- total_sum - Итоговая сумма заказа в рублях. Тип: Числовой (NUMERIC), вычисляемый на основе состава заказа.

Сущность: Состав заказа (OrderInfo)

- order_info_id - Уникальный идентификатор строки состава. Тип: Целочисленный, автоинкрементный.
- order_id - Ссылка на заказ. Тип: Целочисленный, обязательный.
- service_id - Ссылка на услугу. Тип: Целочисленный, обязательный.
- quantity - Количество единиц данной услуги в заказе. Тип: Целочисленный, обязательный, >0.

Сущность: Пользователь системы (User)

- user_id - Уникальный идентификатор пользователя. Тип: Целочисленный, автоинкрементный.
- username - Логин для входа. Тип: Текстовый, обязательный, уникальный.
- password_hash - Хэш пароля (алгоритм SHA-256). Тип: Текстовый, обязательный.
- role - Роль в системе. Тип: Текстовый, обязательный, допустимые значения: 'owner', 'administrator', 'worker', 'accountant'.
- name - Отображаемое имя. Тип: Текстовый.
- created_at - Дата и время создания учетной записи. Тип: Временная метка (TIMESTAMP), заполняется автоматически.

2.3. Определение отношений и кардинальности между объектами

Кардинальность определяет, сколько экземпляров одной сущности может быть связано с экземпляром другой сущности.

Клиент - Заказ (1 : N)

Описание: Один клиент может сделать множество заказов. Каждый заказ принадлежит ровно одному клиенту.

Обоснование: Клиент возвращается в студию многократно. Каждое обращение оформляется как новый заказ.

Обозначение: Client(1) - (N)Order

Должность - Сотрудник (1 : N)

Описание: На одной должности может работать несколько сотрудников. Каждый сотрудник имеет ровно одну основную должность.

Обоснование: В штате может быть несколько фотографов или администраторов.

Обозначение: Post(1) - (N)Employee

Способ оплаты - Заказ (1 : N)

Описание: Одним способом оплаты (например, "банковская карта") можно оплатить множество заказов. Каждый заказ оплачивается одним выбранным способом.

Обозначение: PaymentMethod(1) - (N)Order

Способ связи — Заказ (1 : N)

Описание: Одним способом связи (например, "Telegram") можно оформить множество заказов. Для каждого заказа указывается один предпочтительный способ связи.

Обозначение: ContactMethod(1) — (N)Order

Заказ - Состав заказа (1 : N)

Описание: Один заказ может состоять из нескольких позиций (услуг). Каждая позиция принадлежит ровно одному заказу.

Обоснование: Клиент может заказать одновременно, например, фотосессию и ретушь нескольких кадров.

Обозначение: Order(1) - (N)OrderInfo

Услуга - Состав заказа (1 : N)

Описание: Одна и та же услуга (например, "часовая съемка") может входить в множество заказов. Каждая позиция в составе заказа ссылается на одну конкретную услугу.

Обозначение: Service(1) - (N)OrderInfo

Заказ - Услуга (M : N через Состав заказа)

Фактическая связь: Многие ко многим. Один заказ может включать несколько услуг, и одна услуга может быть в нескольких заказах.

Реализация: Данная связь не может быть выражена напрямую в реляционной модели и реализуется через промежуточную сущность "Состав

заказа" (Order_Info), которая содержит внешние ключи на обе сущности (order_id, service_id) и дополнительный атрибут quantity.

Обозначение: Order(M) — (N)Service

2.4. Построение концептуальной модели

Ниже представлена текстовая ER-диаграмма (Entity-Relationship), отображающая сущности, их атрибуты и связи.

Сущности:

1. Post (Должности)

└─ post_id (PK)

└─ name

2. Employee (Сотрудники)

└─ employee_id (PK)

└─ surname

└─ name

└─ patronymic

└─ post_id (FK → Post.post_id)

└─ phone

3. Client (Клиенты)

└─ client_id (PK)

└─ surname

└─ name

└─ patronymic

└─ phone

└─ email

4. Users (Пользователи системы)

└─ user_id (PK)

└─ username

└─ password_hash

└─ role

└─ name

└─ created_at

5. Orders (Заказы)

- |— order_id (PK)
- |— client_id (FK → Client.client_id)
- |— payment_method_id (FK → PaymentMethod.payment_method_id)
- |— contact_method_id (FK → ContactMethod.contact_method_id)
- |— date
- └— total_sum

6. PaymentMethod (Способы оплаты)

- |— payment_method_id (PK)
- └— name

7. ContactMethod (Способы связи)

- |— contact_method_id (PK)
- └— name

8. OrderInfo (Состав заказов)

- |— order_info_id (PK)
- |— order_id (FK → Orders.order_id)
- |— service_id (FK → Service.service_id)
- └— quantity

9. Service (Услуги)

- |— service_id (PK)
- |— name
- └— service_price

СВЯЗИ:

1. Post(1) - Employee(N)
2. Client(1) - Orders(N)
3. PaymentMethod(1) - Orders(N)
4. ContactMethod(1) - Orders(N)
5. Orders(1) - OrderInfo(N)

6. Service(1) - OrderInfo(N) 7. Orders(M) - Service(N) **Ключевые выводы по концептуальной модели:**

Центральность Order: Сущность Order является связующим звеном между клиентом (Client) и услугами (Service).

Нормализация: Модель избегает избыточности. Например, цена услуги хранится только в Service, а не дублируется в каждом Order_Info.

Гибкость: Связь Order - Service типа M:N через Order_Info позволяет формировать сложные заказы с переменным составом и количеством услуг.

Модульность: Справочные сущности (Post, Payment_Method, Contact_Method, Service) выделены отдельно, что упрощает их поддержку и изменение.

Отделение бизнес-логики от безопасности: Сущность User существует независимо от Employee, что позволяет гибко управлять доступом (например, одна учетная запись на несколько сотрудников или наоборот).

Данная концептуальная модель является полным и непротиворечивым описанием информационных потребностей фотостудии и служит основой для проектирования логической и физической структуры базы данных.

Глава 3

3. Логическая структура БД

Логическая модель представляет собой преобразование концептуальной модели в схему реляционной базы данных. На этом этапе сущности становятся таблицами, атрибуты - столбцами, а связи реализуются через механизмы первичных и внешних ключей.

3.1. Преобразование сущностей в таблицы

Каждая сущность из концептуальной модели преобразуется в таблицу со следующими свойствами:

Таблица post (Должности)

Назначение: Справочник должностей сотрудников.

Столбцы:

- post_id (SERIAL, PRIMARY KEY) — уникальный идентификатор.

- name (TEXT, NOT NULL, UNIQUE) — название должности.

Таблица employee (Сотрудники) Назначение:

Учет работников фотостудии.

Столбцы:

- employee_id (SERIAL, PRIMARY KEY)
- surname (TEXT, NOT NULL)
- name (TEXT, NOT NULL)
- patronymic (TEXT)
- post_id (INT, REFERENCES post(post_id)) — связь с должностью.
- phone (VARCHAR(255))

Таблица client (Клиенты) Назначение:

База данных клиентов.

Столбцы:

- client_id (SERIAL, PRIMARY KEY)
- surname (TEXT)
- name (TEXT, NOT NULL)
- patronymic (TEXT)
- phone (VARCHAR(255))
- email (TEXT)

Таблица payment_method (Способы оплаты)

Назначение: Справочник вариантов оплаты. **Столбцы:**

- payment_method_id (SERIAL, PRIMARY KEY)
- name (TEXT, NOT NULL, UNIQUE)

Таблица contact_method (Способы связи) Назначение:

Справочник каналов коммуникации.

Столбцы:

- contact_method_id (SERIAL, PRIMARY KEY)
- name (TEXT, NOT NULL, UNIQUE)

Таблица service (Услуги)

Назначение: Каталог услуг с ценами.

Столбцы:

- service_id (SERIAL, PRIMARY KEY)
- name (TEXT, NOT NULL)
- service_price (NUMERIC(10,2), NOT NULL)

Таблица orders (Заказы)

Назначение: Регистрация всех совершённых заказов.

Столбцы:

- order_id (SERIAL, PRIMARY KEY)
- client_id (INT, NOT NULL, REFERENCES client(client_id))
- payment_method_id (INT, NOT NULL, REFERENCES payment_method(payment_method_id))
- contact_method_id (INT, NOT NULL, REFERENCES contact_method(contact_method_id)) - date (TIMESTAMP DEFAULT NOW())
- total_sum (NUMERIC(10,2) DEFAULT 0)

Таблица order_info (Состав заказов)

Назначение: Детализация заказов (связь M:N между orders и service).

Столбцы:

- order_info_id (SERIAL, PRIMARY KEY)

- order_id (INT, NOT NULL, REFERENCES orders(order_id) ON DELETE CASCADE)
- service_id (INT, NOT NULL, REFERENCES service(service_id))
- quantity (INT, NOT NULL, CHECK (quantity > 0))

Таблица users (Пользователи системы)

Назначение: Управление доступом к приложению.

Столбцы:

- user_id (SERIAL, PRIMARY KEY)
- username (TEXT, NOT NULL, UNIQUE)
- password_hash (TEXT, NOT NULL)
- role (TEXT, NOT NULL, CHECK (role IN ('owner', 'administrator', 'worker', 'accountant')))
- name (TEXT)
- created_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

3.2. Реализация связей между таблицами

Связи из концептуальной модели реализованы через ограничения внешнего ключа (FOREIGN KEY):

Связь (Концептуальная)	Реализация в БД	Тип связи в БД	Примечание
Post(1) - Employee(N)	employee.post_id → post.post_id	1:N	Каждый сотрудник ссылается на одну должность
Client(1) - Order(N)	orders.client_id → client.client_id	1:N	Каждый заказ ссылается на одного клиента
PaymentMethod(1) - Order(N)	orders.payment_method_id → payment_method.payment_method_id	1:N	Связь заказа со способом оплаты
ContactMethod(1) - Order(N)	orders.contact_method_id → contact_method.contact_method_id	1:N	Связь заказа со способом связи
Order(1) - OrderInfo(N)	order_info.order_id → orders.order_id	Текст в интерфейсе	1:N
Service(1) - OrderInfo(N)	order_info.service_id → service.service_id	1:N	Указание услуги в составе заказа

Order(M) - Service(N)	order_info.service_id → service.service_id	M:N	Косвенная связь через промежуточную таблицу
-----------------------	--	-----	---

3.3. Нормализация базы данных

База данных приведена к третьей нормальной форме (3NF):

1. Первая нормальная форма (1NF):

- Все таблицы имеют первичный ключ
- Все значения атомарны (нет составных полей)
- В каждой ячейке только одно значение
- Нет повторяющихся групп данных

2. Вторая нормальная форма (2NF):

- Соответствует 1NF
- Все неключевые атрибуты полностью зависят от первичного ключа
- В таблице order_info составной ключ мог бы быть (order_id, service_id), но введен суррогатный ключ order_info_id для упрощения

3. Третья нормальная форма (3NF):

- Соответствует 2NF
- Отсутствуют транзитивные зависимости между неключевыми атрибутами - Все справочники выделены в отдельные таблицы

3.4. Ограничения целостности

Для обеспечения целостности данных определены следующие ограничения:

- **Первичные ключи (PRIMARY KEY):** Гарантируют уникальность каждой записи.
- **Внешние ключи (FOREIGN KEY):** Обеспечивают ссылочную целостность.

Проверочные ограничения (CHECK):

- **order_info.quantity > 0** - количество не может быть отрицательным или нулевым
- **service.service_price >= 0** - цена услуги не может быть отрицательной
- **users.role IN (...)** - ограничение допустимых ролей
- **Ограничения NOT NULL:** Для обязательных полей.
- **Ограничения UNIQUE:** Для уникальных значений (post.name, payment_method.name и др.).

3.5. Примеры запросов, демонстрирующие логическую структуру

```
SELECT o.order_id, o.date, c.surname || ' ' || c.name as client_name, pm.name as  
payment_method, cm.name as contact_method, o.total_sumFROM orders oJOIN  
client c ON o.client_id = c.client_idJOIN payment_method pm ON  
o.payment_method_id = pm.payment_method_idJOIN contact_method cm ON  
o.contact_method_id = cm.contact_method_idWHERE o.order_id = 1;
```

```
SELECT s.name as service_name, oi.quantity, s.service_price, (oi.quantity *  
s.service_price) as sum_for_serviceFROM order_info oiJOIN service s ON  
oi.service_id = s.service_idWHERE oi.order_id = 1;
```

```
SELECT c.client_id, c.surname || ' ' || c.name as client_name, c.phone,  
COUNT(o.order_id) as orders_count, SUM(o.total_sum) as total_spentFROM  
client cLEFT JOIN orders o ON c.client_id = o.client_idGROUP BY c.client_id,  
c.surname, c.name, c.phone;
```

3.6. Преимущества выбранной логической структуры

- **Гибкость:** Возможность легко добавлять новые услуги, способы оплаты и связи без изменения структуры основных таблиц.
- **Масштабируемость:** Структура поддерживает рост данных — тысячи клиентов и заказов.
- **Производительность:** Нормализованная структура уменьшает избыточность данных и уменьшает объем хранимой информации.

- **Целостность:** Система внешних ключей и ограничений предотвращает попадание в БД некорректных данных.
- **Адаптивность:** Структура позволяет в будущем добавлять новые атрибуты к существующим сущностям (например, дату рождения клиента, скидочную карту и т.д.).

Логическая структура БД полностью соответствует требованиям предметной области и готова для преобразования в физическую модель с учетом особенностей выбранной СУБД (PostgreSQL).

Глава 4

4. Физическая структура базы данных

Физическая модель определяет конкретную реализацию логической структуры в выбранной СУБД PostgreSQL с учетом требований к производительности, надежности и безопасности.

4.1. Выбор и обоснование СУБД

Выбранная СУБД: PostgreSQL 15

Обоснование выбора: Открытая лицензия, оптимизатор запросов, богатые встроенные типы, репликация, шардинг, активное сообщество, поддержка

Итог: PostgreSQL выбран как наиболее надежная, безопасная и функциональная СУБД для бизнес-приложения.

4.2. Параметры базы данных

```
CREATE DATABASE photo_studio;
```

4.3. Типы данных и их обоснование Критерии выбора типов данных:

- **Точность** - для финансовых расчетов
- **Производительность** - для частых операций
- **Безопасность** - для предотвращения переполнения
- **Совместимость** - с клиентскими приложениями

Таблица соответствия типов данных:

Логический тип	Физический тип (PostgreSQL)	Обоснование	Пример
Уникальный идентификатор	SERIAL или BIGSERIAL	Автоинкремент, 4 байта (SERIAL) или 8 байт (BIGSERIAL)	client_id SERIAL PRIMARY KEY
ФИО, названия	TEXT	Неограниченная длина, эффективное хранение	surname TEXT NOT NULL
Телефон, короткие коды	VARCHAR(255)	Ограничение длины, индексация	phone VARCHAR(255)
Денежные суммы	NUMERIC(10,2)	Точность 2 знака после запятой, без ошибок округления	service_price NUMERIC(10,2)
Количество, целые числа	INTEGER	4 байта, диапазон ± 2.1 млрд	quantity INTEGER

Даты и время	TIMESTAMP WITH TIME ZONE	Учет часовых поясов, точность до микросекунд	date TIMESTAMPTZ DEFAULT NOW()
Хэши паролей	TEXT	Длина SHA-256 хэша 64 символа	password_hash TEXT NOT NULL
Перечисляемые значения	TEXT с CHECK constraint	Простота, читаемость	role TEXT CHECK (role IN (...))

4.4. Стратегия именования Принципы

именования:

- **Таблицы:** имена в единственном числе, snake_case
- **Столбцы:** snake_case, понятные названия
- **Первичные ключи:** (table_name)_id
- **Внешние ключи:** (referenced_table_name)_id
- **Индексы:** idx_(table_name)_(columns)
- **Ограничения:** (table_name)(column_name)(constraint_type) **Примеры:**
- CREATE TABLE client (...);
- client_id - client, а не client_id - clients
- CREATE TABLE order_info (...);
- order_info_id - order_info, а не order info - order infos
- CREATE INDEX idx_orders_client_id_date ON orders(client_id, date);
- ALTER TABLE order_info ADD CONSTRAINT order_info_quantity_positive CHECK (quantity > 0);

4.5. Оптимизация производительности

1. Стратегия индексирования:

- CREATE INDEX idx_orders_date_client_total ON orders(date, client_id, total_sum);
- CREATE INDEX idx_order_info_service_quantity ON order_info(service_id, quantity);
- CREATE INDEX idx_clients_name_phone ON clients(surname, name, phone);

- CREATE INDEX idx_orders_recent ON orders(date) WHERE date > CURRENT_DATE - INTERVAL '90 days';

2. Партиционирование (для больших объемов данных):

- CREATE TABLE orders () PARTITION BY RANGE (date);
- CREATE TABLE orders_2024_01 PARTITION OF orders FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

3. Материализованные представления для отчетов:

- CREATE MATERIALIZED VIEW mv_daily_stats AS SELECT DATE(date) as report_date, COUNT(*) as orders_count, SUM(total_sum) as total_revenue, COUNT(DISTINCT client_id) as unique_clients FROM orders GROUP BY DATE(date);

4.6. Настройки безопасности

1. Шифрование соединений:

- ssl = on ssl_cert_file = 'server.crt' ssl_key_file = 'server.key'

2. Аудит и мониторинг:

- log_statement = 'all' log_connections = on log_disconnections = on
- CREATE EXTENSION pgaudit;
- SET pgaudit.log = 'all, -misc';

3. Безопасность на уровне строк (RLS):

- ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
- CREATE POLICY accountant_policy ON orders FOR SELECT TO accountant_user USING (EXTRACT(YEAR FROM date) = EXTRACT(YEAR FROM CURRENT_DATE));

4.7. Стратегия хранения

1. Tablespace для разных типов данных:

- CREATE TABLESPACE fast_ssd LOCATION '/ssd/postgresql/data';
- CREATE TABLESPACE archive_hdd LOCATION '/hdd/postgresql/archive'; -
- CREATE TABLE orders (...) TABLESPACE fast_ssd;

2. Настройка autovacuum:

- autovacuum_vacuum_scale_factor = 0.1autovacuum_analyze_scale_factor = 0.05autovacuum_vacuum_cost_delay = 10ms

4.8. Мониторинг и обслуживание

Ключевые метрики для мониторинга:

- **Размер БД:** SELECT pg_size_pretty(pg_database_size('photo_studio'));
- **Активные подключения:** SELECT count() FROM pg_stat_activity;*
- **Эффективность кэша:** SELECT blks_hit*100/(blks_hit+blks_read) as cache_hit_ratio FROM pg_stat_database WHERE datname='photo_studio';
- **Медленные запросы:** через pg_stat_statements **План обслуживания:**
- **Ежедневно:** Проверка логов ошибок, мониторинг места на диске
- **Еженедельно:** Анализ медленных запросов, обновление статистики
- **Ежемесячно:** Переиндексация фрагментированных индексов, проверка целостности
- **Ежеквартально:** Анализ роста данных, планирование масштабирования

4.9. Резервное копирование и восстановление -

Многоуровневая стратегия:

archive_mode = onarchive_command = 'cp %p /backup/wal_archive/%f'

pg_dump -Fc -b -v -f /backup/full_\$(date +%Y%m%d).dump

photo_studio pg_basebackup -D /backup/monthly/\$(date +%Y%m) -Ft -z

- **Процедура восстановления:** pg_restore -C -d postgres

/backup/full_20241215.dump recovery_target_time = '2024-12-15

14:30:00'

4.10. Миграция и обновление

План миграции при изменении схемы:

- Создание бэкапа перед изменениями
- Использование транзакций для атомарности изменений
- Версионирование миграционных скриптов
- Тестирование на staging-окружении
- Откат при обнаружении проблем

Пример миграционного скрипта:

- BEGIN;
- ALTER TABLE client ADD COLUMN discount_card_number VARCHAR(20) DEFAULT NULL;
- CREATE INDEX idx_clients_discount_card ON client(discount_card_number);
- COMMIT;

Физическая структура спроектирована с учетом текущих и будущих потребностей фотостудии, обеспечивая баланс между производительностью, надежностью и простотой поддержки.

Глава 5

5. Реализация проекта в среде конкретного СУБД

5.1. Создание таблиц

- CREATE TABLE post (post_id SERIAL PRIMARY KEY, name TEXT NOT NULL UNIQUE);
- CREATE TABLE employee (employee_id SERIAL PRIMARY KEY, surname TEXT NOT NULL, name TEXT NOT NULL, patronymic text, post_id INT REFERENCES post(post_id), phone VARCHAR(255));
- CREATE TABLE client (client_id SERIAL PRIMARY KEY, surname TEXT, name TEXT NOT NULL, patronymic text, phone VARCHAR(255), email TEXT);

- CREATE TABLE payment_method (payment_method_id SERIAL PRIMARY KEY, name TEXT NOT NULL UNIQUE);
- CREATE TABLE contact_method (contact_method_id SERIAL PRIMARY KEY, name TEXT NOT NULL UNIQUE);
- CREATE TABLE service (service_id SERIAL PRIMARY KEY, name TEXT NOT NULL, service_price NUMERIC(10, 2) NOT NULL);
- CREATE TABLE order_info (order_info_id SERIAL PRIMARY KEY, order_id INT, service_id INT REFERENCES service(service_id), quantity INT);
- CREATE TABLE orders (order_id SERIAL PRIMARY KEY, client_id INT REFERENCES client(client_id), payment_method_id INT REFERENCES payment_method(payment_method_id), date TIMESTAMP DEFAULT NOW(), total_sum NUMERIC(10, 2), contact_method_id INT REFERENCES contact_method(contact_method_id));
- CREATE TABLE users (user_id SERIAL PRIMARY KEY, username TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL, role TEXT NOT NULL CHECK (role IN ('owner', 'administrator', 'worker', 'accountant')), name TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

5.2. Создание запросов

- SELECT COUNT(*) as orders_count, SUM(total_sum) as total_revenue, COUNT(DISTINCT client_id) as unique_clients FROM orders WHERE date::date = %s
- SELECT s.name as service_name, SUM(oi.quantity) as total_quantity, SUM(oi.quantity * s.service_price) as service_revenue FROM orders o JOIN order_info oi ON o.order_id = oi.order_id JOIN service s ON oi.service_id = s.service_id WHERE o.date::date = %s GROUP BY s.service_id, s.name ORDER BY service_revenue DESC
- SELECT o.order_id, c.surname || ' ' || c.name as client_name, o.total_sum, pm.name as payment_method, cm.name as contact_method, o.date FROM orders o JOIN client c ON o.client_id = c.client_id JOIN payment_method pm ON o.payment_method_id = pm.payment_method_id JOIN contact_method cm ON o.contact_method_id = cm.contact_method_id WHERE o.date::date = %s ORDER BY o.date

- SELECT o.order_id, c.surname || ' ' || c.name as client, s.name as service, oi.quantity, s.service_price, (oi.quantity * s.service_price) as total, pm.name as payment_method, cm.name as contact_method, o.date FROM orders o JOIN client c ON o.client_id = c.client_id JOIN order_info oi ON o.order_id = oi.order_id JOIN service s ON oi.service_id = s.service_id JOIN payment_method pm ON o.payment_method_id = pm.payment_method_id JOIN contact_method cm ON o.contact_method_id = cm.contact_method_id WHERE o.date::date BETWEEN %s AND %s ORDER BY o.date DESC
- SELECT s.name, COUNT(*), SUM(oi.quantity * s.service_price) FROM order_info oi JOIN service s ON oi.service_id = s.service_id GROUP BY s.service_id, s.name ORDER BY SUM(oi.quantity * s.service_price) DESC LIMIT 10
- SELECT c.surname || ' ' || c.name, COUNT(*), SUM(o.total_sum) FROM orders o JOIN client c ON o.client_id = c.client_id GROUP BY c.client_id, c.surname, c.name ORDER BY SUM(o.total_sum) DESC LIMIT 10
- SELECT service_id, name, service_price FROM service WHERE service_id = %s
SELECT surname, name, patronymic, phone, email FROM client WHERE client_id = %s
SELECT name FROM payment_method WHERE payment_method_id = %s
SELECT name FROM contact_method WHERE contact_method_id = %s
SELECT column_name, data_type FROM information_schema.columns WHERE table_name = '{table_name}'
- SELECT COUNT(*) FROM orders
SELECT SUM(total_sum) FROM orders
SELECT table_name FROM information_schema.tables WHERE table_schema = 'public'
- INSERT INTO orders (client_id, payment_method_id, contact_method_id, total_sum, date) VALUES (%s, %s, %s, %s, NOW()) RETURNING order_id;
INSERT INTO order_info (order_id, service_id, quantity) VALUES (%s, %s, %s);
INSERT INTO users (username, password_hash, role, name) VALUES (%s, %s, %s, %s);
INSERT INTO {table_name} VALUES ({', '.join(values)});

5.3. Разработка интерфейса Архитектура

приложения:

УРОВЕНЬ 1: ИСПОЛНЯЕМЫЙ ФАЙЛ

└─ photo_studio.exe - Скомпилированное приложение

УРОВЕНЬ 2: ФУНКЦИОНАЛЬНЫЕ МОДУЛИ (functions)

- |— database.py - Работа с базой данных
- |— auth.py - Аутентификация и авторизация
- |— calculations.py - Расчеты и бизнес-логика
- └─ utils.py - Вспомогательные функции

УРОВЕНЬ 3: КОМПОНЕНТЫ ИНТЕРФЕЙСА

- |— Основные окна:
 - | |— main.py - Точка входа приложения |
 - | └─ main_window.py - Главное окно интерфейса
- |— Диалоговые окна:
 - | |— login_window.py - Окно авторизации
 - | |— order_dialog.py - Работа с заказами
 - | |— client_dialog.py - Управление клиентами
 - | |— report_dialog.py - Формирование отчетов
 - | └─ receipt_dialog.py - Печать чеков └─
- Сервисные модули:
 - └─ export.py - Экспорт данных в файлы

Основные компоненты интерфейса:

1. Окно авторизации (LoginWindow):

- Поля для ввода логина и пароля
- Валидация учетных данных через users таблицу - Запоминание последнего пользователя (опционально)

2. Главное окно (MainWindow):

- Меню с разделами: Файл, Данные, Отчеты, Администрирование
- Панель вкладок, адаптирующаяся под роль пользователя - Статусная строка с информацией о пользователе и подключении

3. Вкладка "Клиенты":

- Таблица со списком клиентов с сортировкой и фильтрацией
- Кнопки: Добавить, Редактировать, Удалить

4. Вкладка "Заказы":

- Таблица текущих заказов
- Форма создания нового заказа с "корзиной" услуг

5. Вкладка "Услуги":

- Редактируемый прайс-лист
- Возможность добавления/удаления/изменения услуг

6. Вкладка "Отчеты":

- Форма выбора периода для отчетов
- Предпросмотр отчетов в табличном виде
- Кнопки экспорта в CSV/Excel/PDF

7. Диалог "Чек":

Предпросмотр чека в формате для печати

Сохранение в CSV/TXT

5.4. Назначение прав доступа

- GRANT CONNECT ON DATABASE photo_studio TO owner_user, dba_user, accountant_user, worker_user;
- GRANT USAGE ON SCHEMA public TO owner_user, dba_user, accountant_user, worker_user;
- GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO owner_user;
- GRANT TEMPORARY ON DATABASE photo_studio TO owner_user;
- ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO owner_user;
- ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT USAGE, SELECT ON SEQUENCES TO owner_user;
- GRANT SELECT ON ALL TABLES IN SCHEMA public TO dba_user;
- GRANT INSERT, UPDATE, DELETE ON post, employee, client, payment_method, contact_method, service, order_info, orders TO dba_user;
- GRANT SELECT ON users TO dba_user;

- GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO dba_user;
- ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO dba_user;
- ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT USAGE, SELECT ON SEQUENCES TO dba_user;
- GRANT SELECT ON service, payment_method, client, post, employee, contact_method TO worker_user;
- GRANT SELECT, INSERT ON orders, order_info, client TO worker_user;
- GRANT UPDATE ON client TO worker_user;
- GRANT USAGE, SELECT ON orders_order_id_seq, order_info_order_info_id_seq, client_client_id_seq TO worker_user;
- GRANT SELECT ON orders, order_info, service, client, payment_method, employee TO accountant_user;
- GRANT SELECT ON orders_order_id_seq, order_info_order_info_id_seq TO accountant_user;

5.5. Создание индексов

- CREATE INDEX idx_orders_date_total ON orders(order_date, total_sum);
- CREATE INDEX idx_service_name ON service(name);CREATE INDEX idx_service_price ON service(service_price);CREATE INDEX idx_service_active ON service(service_id) WHERE is_active = TRUE;
- CREATE INDEX idx_employee_name ON employee(surname, name);CREATE INDEX idx_employee_post ON employee(post_id);CREATE INDEX idx_employee_active ON employee(employee_id) WHERE is_active = TRUE;
- CREATE INDEX idx_users_username ON users(username);CREATE INDEX idx_users_role ON users(role);CREATE INDEX idx_users_active ON users(user_id) WHERE is_active = TRUE;
- CREATE INDEX idx_audit_changed_at ON audit_log(changed_at DESC);CREATE INDEX idx_audit_table_operation ON audit_log(table_name, operation);

5.6. Разработка стратегии резервного копирования баз данных Функция для создания полного бэкапа:

```
- CREATE OR REPLACE FUNCTION create_full_backup()RETURNS TEXTLANGUAGE
plpgsqlAS $$DECLARE  backup_file TEXT;  backup_path TEXT := '/backup/full/';
timestamp_str TEXT := TO_CHAR(CURRENT_TIMESTAMP,
'YYYYMMDD_HH24MISS');BEGIN  backup_file := backup_path ||
'photo_studio_full_' || timestamp_str || '.dump';  INSERT INTO backup_log
(backup_type, filename, size_mb, status)  VALUES ('full', backup_file, NULL,
'started');  RETURN backup_file;END;$$;
```

Функция для создания инкрементального бэкапа:

```
- CREATE OR REPLACE FUNCTION create_incremental_backup()RETURNS
TEXTLANGUAGE plpgsqlAS $$DECLARE  backup_file TEXT;  backup_path TEXT :=
'/backup/incremental/';  timestamp_str TEXT :=
TO_CHAR(CURRENT_TIMESTAMP, 'YYYYMMDD_HH24MISS');BEGIN  IF NOT
EXISTS (    SELECT 1 FROM backup_log    WHERE backup_type = 'incremental'
AND created_at > CURRENT_DATE - INTERVAL '1 day'    AND status =
'completed'  ) THEN    backup_file := backup_path || 'photo_studio_inc_' ||
timestamp_str || '.dump';    INSERT INTO backup_log (backup_type, filename,
size_mb, status)    VALUES ('incremental', backup_file, NULL, 'started');
RETURN backup_file;  ELSE    RETURN NULL;  END IF;END;$$;
```

Функция для получения последнего успешного бэкапа:

```
- CREATE OR REPLACE FUNCTION get_latest_backup(backup_type_param
VARCHAR DEFAULT 'full')RETURNS TABLE (  backup_id INTEGER,  filename TEXT,
start_time TIMESTAMP,  size_mb NUMERIC(10,2))LANGUAGE plpgsqlAS $$BEGIN
RETURN QUERY  SELECT    bl.backup_id,    bl.filename,    bl.start_time,
bl.size_mb  FROM backup_log bl  WHERE bl.backup_type = backup_type_param
AND bl.status = 'completed'  ORDER BY bl.start_time DESC  LIMIT 1;END;$$;
```

Функция для проверки целостности бэкапа:

```
- CREATE OR REPLACE FUNCTION verify_backup(backup_id_param
INTEGER)RETURNS BOOLEANLANGUAGE plpgsqlAS $$DECLARE  backup_file
TEXT;  is_valid BOOLEAN := FALSE;BEGIN  SELECT filename INTO backup_file
FROM backup_log  WHERE backup_id = backup_id_param;  IF backup_file IS
NULL THEN    RAISE EXCEPTION 'Backup not found: %', backup_id_param;  END
```

```
IF; UPDATE backup_log SET status = 'verified', checksum =  
MD5(backup_file) -- упрощенный пример WHERE backup_id =  
backup_id_param; RETURN is_valid;END;$$;
```

5.7. Разработка стратегии защиты баз данных и хранения информации средствами СУБД (настройка ролей и обеспечение безопасности на уровне строк)

- CREATE SCHEMA config;CREATE SCHEMA data;
- CREATE SCHEMA reports;CREATE SCHEMA audit;
- ALTER TABLE post SET SCHEMA config;
- ALTER TABLE employee SET SCHEMA data;
- ALTER TABLE client SET SCHEMA data;
- ALTER TABLE payment_method SET SCHEMA config;
- ALTER TABLE contact_method SET SCHEMA config;
- ALTER TABLE service SET SCHEMA config;
- ALTER TABLE orders SET SCHEMA data;
- ALTER TABLE order_info SET SCHEMA data;
- ALTER TABLE users SET SCHEMA security;
- ALTER TABLE audit_log SET SCHEMA audit;
- ALTER TABLE backup_log SET SCHEMA audit;

5.8. Разработка API, реализующего работу с базой данных внешнего приложения

```
class DatabaseManager: def __init__(self):
```

```
    self.connection = None  
    self.current_user = None  
    self.current_role = None  
    def connect(self):    try:  
        self.connection = psycopg2.connect(  
    database="photo_studio",
```

```

user="postgres",
password="pgadmin",
host="localhost",      port="5432"
    )
    return True
except Exception as e:
    QMessageBox.critical(None, "Ошибка подключения", f"Не удалось
подключиться к базе данных:\n{str(e)}")
    return False

```

```

def login(self, username, password):
try:
    cursor = self.connection.cursor()
    cursor.execute("""
        SELECT user_id, username, password_hash, role, name
        FROM users
        WHERE username = %s
    """, (username,))
    user_data = cursor.fetchone()
    cursor.close()
    if user_data:
        password_hash = hashlib.sha256(password.encode()).hexdigest()
    if user_data[2] == password_hash:
        self.current_user = {
            'id': user_data[0],

```

```

        'username': user_data[1],
        'role': user_data[3],
        'name': user_data[4]
    }

    self.current_role = user_data[3]

return True    except Exception as e:

    print(f"Ошибка при входе: {e}")

return False

```

6. Заключение

В ходе выполнения курсового проекта была разработана и реализована комплексная информационная система для автоматизации бизнес-процессов фотостудии. Проект включал полный цикл разработки — от анализа предметной области до создания рабочего приложения с графическим интерфейсом и безопасной базой данных.

Основные достижения проекта:

1. Полная автоматизация ключевых процессов:

- Разработана нормализованная реляционная база данных на PostgreSQL
- Реализованы все основные бизнес-процессы: управление клиентами, оформление заказов, учет услуг, формирование отчетов
- Создано desktop-приложение на Python с PyQt6 с интуитивно понятным интерфейсом

2. Хороший уровень безопасности:

- Внедрена четырехуровневая ролевая модель доступа (владелец, администратор, работник, бухгалтер)

3. Надежность:

- Разработана стратегия резервного копирования и восстановления данных

4. Гибкость и масштабируемость:

- Создана модульная архитектура приложения с возможностью расширения
- Реализован экспорт данных в различные форматы (CSV, Excel, TXT)

5. Практическая значимость:

- Система готова к внедрению в реальную фотостудию
- Повышение эффективности работы сотрудников

6. Технологический стек:

- СУБД: PostgreSQL 15
- Приложение: Python 3.8+, PyQt6, psycopg2, pandas

7. Перспективы развития:

- Интеграция с онлайн-платежными системами
- Добавление модуля управления расписанием фотографов
- Создание веб-версии для удаленного доступа
- Внедрение системы аналитики на основе машинного обучения

Разработанная система представляет собой законченное решение, которое может быть успешно внедрено в деятельность фотостудий малого и среднего бизнеса. Проект демонстрирует практическое применение знаний в области проектирования баз данных, разработки клиент-серверных приложений и обеспечения информационной безопасности.

7. Список использованных источников

К. Дж. Дейт. Введение в системы баз данных.

Р. Элмасри, Ш. Б. Наватхе. Системы баз данных: полный курс.

М. Дж. Хернандес. Проектирование баз данных. Руководство для начинающих.

Т. Лейн, С. Момджян. PostgreSQL. Основы языка SQL.

Ф. Лучано. PostgreSQL для начинающих.

PostgreSQL 15 Documentation.

Г. Шилдт. Python: полное руководство.

М. Саммерфилд. Программирование на PyQt.

Э. Майклс. Python и PyQt. Разработка приложений.

Python 3.11 Documentation.

PyQt6 Documentation. psycopg2

Documentation.

PostgreSQL Official Website. <https://www.postgresql.org>

Python Official Website. <https://www.python.org>

PyQt Official Website. <https://www.riverbankcomputing.com/software/pyqt/>

Stack Overflow - вопросы и ответы по программированию.
<https://stackoverflow.com>

GitHub — репозитории с примерами кода. <https://github.com>

7. Даты и подписи

Дата составления: _____

Подпись автора: _____

Оценка: _____

Подпись научного руководителя: _____